

Dynamic Tick Scheduling in xv6

Avishkar Kawadgave

1. Implementation Performed and Initial Assessment

1.1 Using usertests and procdump for initial assessment

usertests represents the most exhaustive validation workload used in this study because it stresses a broad range of xv6 functionality, including user-space execution paths, trap handling, memory-protection checks, file-system operations, and process-creation behavior. The screenshots below further support this observation by showing the intensity of **usertests** execution under load.

```
[akawadga@bondi ~/xv6-riscv]$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ forktest
fork test
fork test OK
$ usertests
usertests starting
test copyin: OK
test copyout: OK
test copyinstr1: OK
test copyinstr2: OK
test copyinstr3: OK
test rwsbrk: OK
test truncate1: OK
test truncate2: OK
test truncate3: OK
test openinput: OK
test exitinput: OK
test iput: OK
test opentest: OK
test writetest: OK
test writebig: OK
test createtest: OK
test dirttest: OK
test exectest: OK
test pipel: OK
test killstatus: OK
test preempt: kill... wait... OK
test exitwait: OK
test reparent: OK
test twochildren: OK
test forkfork: OK
test forkforkfork: OK
test reparent2: OK
test mem: OK
test sharedfd: OK
test fourfiles: OK
test createdelete: OK
test unlinkread: OK
test linktest: OK
test concreate: OK
test linkunlink: OK
test subdir: OK
```

Figure 1: **usertests** execution under heavy workload conditions.

Consequently, to observe processor behaviour under a high-pressure workload, **procdump** was used to examine how many processes could be present simultaneously during execution.

The xv6 process states present in the codebase during this assessment are listed below.

Enum State	Printed Label
[UNUSED]	"unused"
[USED]	"used"
[SLEEPING]	"sleep "
[RUNNABLE]	"runble"
[RUNNING]	"run "
[ZOMBIE]	"zombie"

The **procdump** output shown below was captured while **usertests** was running, with the goal of observing the number of processes present during a heavy workload. The snapshot aligns with the process states

listed above and indicates that as many as 10 process entries were observed concurrently during execution. This serves as an initial measure of the process pressure and scheduler contention introduced by the workload.

```
test forkfork: OK
test forkforkfork:
1 sleep  init
2 sleep  sh
3 sleep  usertests
3041 sleep  usertests
3042 run    usertests
3043 sleep  usertests
3044 sleep  usertests
3045 sleep  usertests
3046 sleep  usertests
3047 sleep  usertests
3048 sleep  usertests
3049 sleep  usertests
3050 sleep  usertests
3051 sleep  usertests
3052 sleep  usertests
3053 sleep  usertests
3054 sleep  usertests
3055 sleep  usertests
3056 sleep  usertests
3057 sleep  usertests
3058 sleep  usertests
3059 sleep  usertests
OK
test reparent2:
1 run    init
2 sleep  sh
3 sleep  usertests
3102 runble usertests
3443 zombie usertests
3444 zombie usertests
3445 zombie usertests
3446 run    usertests
```

Figure 2: procdump output during the heavy-workload in usertests execution.

1.2 Instrumentation added for scheduler reporting

To compare the baseline round-robin scheduler against the adaptive version, additional instrumentation was required. In particular, the kernel needed to expose two classes of measurements back to user space:

1. the number of context switches observed during execution, and
2. the execution time measured in ticks for the benchmark being run.

Since these metrics originate in kernel space but need to be reported from the benchmark programs, the changes were integrated through the syscall path.

Layer	Files modified
Kernel space	<code>proc.c</code> , <code>syscall.c</code> , <code>syscall.h</code> , <code>sysproc.c</code> , <code>trap.c</code>
User space	<code>forktest.c</code> , <code>user.h</code> , <code>usertests.c</code> , <code>usys.pl</code> , <code>ls.c</code>

Table 1: Files updated to expose scheduler metrics from kernel space to user-space benchmarks.

This integration keeps the measurement path simple: the scheduler maintains the counters, the syscall path exposes them, and the benchmark programs print the final values for repeated runs.

The implementation screenshots are included in Appendix A.

2. Baseline Metrics for Round-Robin Execution

2.1 Baseline `usertests` Execution

The first baseline dataset collected was for `usertests` under the default round-robin scheduler. The table below records ten runs and the average across those runs. The “proc diff” quantities correspond to the observed differences in the counts of processes seen in each state during execution.

Table 2: Baseline Round Robin Scheduling for `usertests` Execution

Run	Running proc diff	Sleeping proc diff	Runnable proc diff	Unused proc diff	Used proc diff	Zombie proc diff	Execution Time (ticks)	Total Context Switches
1	30286	489179	94435	7576883	172	5194	5611	94435
2	31421	484418	94835	7527084	165	1668	6246	94835
3	49520	620691	99019	9537606	192	2358	6276	99018
4	43214	568714	99709	8802090	159	7955	6034	99709
5	46780	608665	94214	9023018	168	707	5136	94214
6	48036	595453	96010	9163427	178	1001	6201	96009
7	41239	563176	99387	8611034	192	788	5984	99387
8	33401	511555	98176	7955491	159	1940	6029	98176
9	47238	591802	95432	8901850	190	1381	6368	95432
10	49774	623069	98522	9405993	168	7499	5982	98520
Avg.	42090.9	565672.2	96973.9	8650447.6	174.3	3049.1	5986.7	96973.5

2.2 Baseline `ls` Execution

Table 3: Baseline Round Robin Scheduling for `ls` Execution

Run	Running proc diff	Sleeping proc diff	Runnable proc diff	Unused proc diff	Used proc diff	Zombie proc diff	Execution Time (ticks)	Total Context Switches
1	124	252	2	7664	0	0	2	2
2	573	1150	2	35029	0	0	2	2
3	132	266	2	8130	0	0	2	2
4	162	328	2	9964	0	0	2	2
5	282	568	2	17293	0	0	2	2
6	166	335	2	10212	0	0	2	2
7	165	334	2	10155	0	0	2	2
8	109	222	2	6746	0	0	2	2
9	79	162	2	4919	0	0	2	2
10	82	168	2	5098	0	0	2	2
Avg.	187.4	378.5	2	11521	0	0	2	2

`ls` stays very light, with only about 2 runnable processes and around 2 ticks and 2 context switches on average. This confirms that `ls` acts as a low-contention workload and provides a useful contrast against the heavier `usertests` workload.

2.3 Baseline forktest Execution

Table 4: Baseline Round Robin Scheduling for **forktest** Execution

Run	Running proc diff	Sleeping proc diff	Runnable proc diff	Unused proc diff	Used proc diff	Zombie proc diff	Execution Time (ticks)	Total Context Switches
1	2	8	63	24	0	159	1	63
2	2	8	63	35	0	148	1	63
3	47	36	64	861	0	141	1	64
4	50	8	63	116	0	19	1	63
5	2	8	63	14	0	168	1	63
6	94	30	62	215	12	547	1	62
7	34	8	63	104	0	47	1	63
8	94	44	62	582	14	612	1	62
9	3	12	64	251	1	50	2	64
10	0	4	63	61	0	0	1	63
Avg.	32.8	16.6	63	226.3	2.7	189.1	1.1	63

forktest keeps the runnable load high, with about 63 runnable processes and nearly 63 context switches on average. Compared to **ls**, the execution time in ticks stays low because **forktest** does not do much computation inside each process. It mostly creates child processes and finishes quickly, so it stresses process creation and scheduling more than raw CPU time.

3. Dynamic Algorithm Decision and Code Block

The **usertests** results show that the workload can put high pressure on the runnable queue. So, instead of keeping one fixed tick interval, this project uses a dynamic tick policy that changes based on system activity, with the aim of reducing ticks while making sure one process does not keep the CPU for too long.

Listing 1: Dynamic tick logic inserted into the scheduler.

```
if(p->state == RUNNABLE) {
    uint64 run_started_at = r_time();
    p->state = RUNNING;
    c->proc = p;
    swtch(&c->context, &p->context);
    total_context_switches++;
    uint64 run_time = r_time() - run_started_at;
    if (run_time > dynamic_tick_rate &&
        proc_runnable_current > 10 &&
        dynamic_tick_rate > 250000) {
        uint64 next_tick_rate = (dynamic_tick_rate * 3) / 4;
        dynamic_tick_rate = next_tick_rate < 250000 ? 250000 : next_tick_rate;
    }
    else if (proc_runnable_current <= 2) {
        if (dynamic_tick_rate < 1000000 && proc_runnable_current == 2) {
            uint64 next_tick_rate = (dynamic_tick_rate * 5) / 4;
            dynamic_tick_rate = next_tick_rate > 1000000 ? 1000000 : next_tick_rate;
        } else {
            dynamic_tick_rate = 1500000;
        }
    }
    c->proc = 0;
    found = 1;
}
release(&p->lock);
```

The adaptive rule uses three values: the runtime of the process that just executed, the current runnable-

process count, and the current dynamic tick rate. The base tick interval is 1500000, and the minimum floor allowed value is 250000. Whenever runnable pressure increases, the tick interval is dropped by 25%. When the runnable-process count decreases, the tick interval is gradually increased again by 25%, so the scheduler becomes less aggressive under lighter workloads.

4. Dynamic Tick Metrics for `usertests`, `ls`, and `forktest`

4.1 Dynamic Tick `usertests` Execution

Table 5: Dynamic Tick Scheduling for `usertests` Execution

Run	Running proc diff	Sleeping proc diff	Runnable proc diff	Unused proc diff	Used proc diff	Zombie proc diff	Execution Time (ticks)	Total Context Switches
1	34394	501278	94078	7838594	133	4102	3804	94078
2	36145	531910	97644	8133524	103	428	4241	97645
3	39190	564127	96599	8432688	172	644	4226	96599
4	36656	520397	94184	8048556	98	3887	4202	94184
5	37762	533415	98628	8345052	127	9970	4296	98628
6	44169	586258	98778	8927240	130	1139	4216	98778
7	36406	517291	96326	8113057	99	350	4205	96325
8	36320	524970	96545	8207229	126	6959	4272	96545
9	41407	574283	98091	8799478	102	1242	4376	98090
10	36650	520382	94578	8087970	126	4756	4394	94578
Avg.	37909.9	537431.1	96545.1	8293338.8	121.6	3347.7	4223.2	96545

4.2 Dynamic Tick `ls` Execution

Table 6: Dynamic Tick Scheduling for `ls` Execution

Run	Running proc diff	Sleeping proc diff	Runnable proc diff	Unused proc diff	Used proc diff	Zombie proc diff	Execution Time (ticks)	Total Context Switches
1	198	407	4	12335	0	0	3	4
2	75	154	2	4673	0	0	2	2
3	262	530	3	16161	0	0	3	3
4	290	584	2	17802	0	0	2	2
5	400	806	3	24578	0	0	3	3
6	151	308	3	9371	0	0	3	3
7	87	180	3	5490	0	0	3	3
8	155	316	3	9613	0	0	3	3
9	240	483	2	14719	0	0	2	2
10	20	44	2	1340	0	0	2	2
Avg.	187.8	381.2	2.7	11608.2	0	0	2.6	2.7

4.3 Dynamic Tick `forktest` Execution

Table 7: Dynamic Tick Scheduling for `forktest` Execution

Run	Running proc diff	Sleeping proc diff	Runnable proc diff	Unused proc diff	Used proc diff	Zombie proc diff	Execution Time (ticks)	Total Context Switches
1	0	2	62	0	0	0	0	62
2	0	2	62	0	0	0	0	62
3	0	2	62	0	0	0	0	62
4	6	16	63	425	0	2	1	63
5	4	10	62	242	0	0	0	62
6	34	18	62	358	0	102	1	62
7	0	2	62	0	0	0	0	62
8	0	2	62	0	0	0	0	62
9	6	16	63	276	0	151	1	63
10	2	4	61	8	0	53	1	61
Avg.	5.2	7.4	62.1	130.9	0	30.8	0.4	62.1

5. Comparison of Average Results and Discussion

Table 8: Average baseline static-tick and dynamic-tick scheduling metrics across workloads

	Running proc diff	Sleeping proc diff	Runnable proc diff	Unused proc diff	Used proc diff	Zombie proc diff	Execution Time (ticks)	Total Context Switches
usertests								
Static Tick Avg.	42090.9	565672.2	96973.9	8650447.6	174.3	3049.1	5986.7	96973.5
Dynamic Tick Avg.	37909.9	537431.1	96545.1	8293338.8	121.6	3347.7	4223.2	96545
1s								
Static Tick Avg.	187.4	378.5	2	11521	0	0	2	2
Dynamic Tick Avg.	187.8	381.2	2.7	11608.2	0	0	2.6	2.7
forktest								
Static Tick Avg.	32.8	16.6	63	226.3	2.7	189.1	1.1	63
Dynamic Tick Avg.	5.2	7.4	62.1	130.9	0	30.8	0.4	62.1

The baseline and dynamic tick results show that the adaptive scheduler does not affect all workloads uniformly. For **usertests**, the dynamic tick interval produces the clearest overall improvement. The average execution time drops from 5986.7 ticks to 4223.2 ticks, which is a 29.5% reduction, while the total context switches only decrease marginally from 96973.5 to 96545, or about 0.44%. At the same time, the running-process difference decreases by 9.9%, the sleeping-process difference decreases by 5.0%, the unused-process difference decreases by 4.1%, and the used-process difference decreases by 30.2%. This indicates that the gain is not coming from a major reduction in runnable pressure, since the runnable-process difference also changes only slightly by about 0.44%, but rather from reducing time spent cycling processes across active states and lowering scheduler overhead during a long and process-heavy workload. One exception is the zombie-process difference, which increases by about 9.8%, showing that not every metric improves simultaneously under the adaptive policy.

For **1s**, the dynamic tick mechanism does not provide any real benefit. This workload is already extremely short, and under such low scheduler pressure the baseline scheduler is effectively sufficient. The average execution time increases from 2 to 2.6 ticks, which is a 30% increase, and the total context switches increase from 2 to 2.7, or 35%. The runnable-process difference also rises by 35%, while the sleeping-process and unused-process differences increase slightly by about 0.7% each. The running-process difference is almost unchanged, increasing by only 0.2%. These results suggest that for very small workloads, the control overhead of adapting the tick interval can outweigh any possible scheduling advantage.

For **forktest**, however, the dynamic tick scheduler shows a much stronger advantage. The average execution time decreases from 1.1 ticks to 0.4 ticks, corresponding to a 63.6% reduction. In parallel, the running-process difference drops from 32.8 to 5.2, an 84.1% decrease, the sleeping-process difference drops by 55.4%, the unused-process difference drops by 42.2%, and the zombie-process difference drops sharply from 189.1 to 30.8, which is an 83.7% reduction. The used-process difference also falls from 2.7 to 0. Although the runnable-process difference and total context switches stay almost the same, both decreasing by only about 1.4%, the much lower execution time shows that the scheduler is handling rapid process creation and termination more efficiently. This is important for **forktest**, where many processes become runnable or terminate within a short time window.

Overall, the current results suggest that the proposed dynamic tick interval algorithm is more effective for workloads with higher process activity, more state transitions, and greater scheduler pressure. In small workloads such as **1s**, the baseline round robin scheduler is already sufficient, so adaptation adds overhead without improving performance. In larger workloads such as **usertests** and process-intensive workloads such as **forktest**, the dynamic tick mechanism reduces execution time while also lowering several process-state difference metrics, especially the running, sleeping, used, and zombie categories. This suggests that the adaptive policy is most useful when the scheduler must react to sustained or bursty concurrency rather than trivial short-lived tasks.

A. Appendix: Implementation Screenshots

```

76 int
77 main(int argc, char *argv[])
78 {
79     // ...
80     // ...
81     // ...
82     // ...
83     // ...
84     // ...
85     // ...
86     // ...
87     // ...
88     // ...
89     // ...
90     // ...
91     // ...
92     // ...
93     // ...
94     // ...
95     // ...
96     // ...
97     // ...
98     // ...
99     // ...
100     // ...
101     // ...
102     // ...
103     // ...
104     // ...
105     // ...
106     // ...
107     // ...

```

(a) ls.c changes

```

1093 int
1094 main(int argc, char *argv[])
1095 {
1096     // ...
1097     // ...
1098     // ...
1099     // ...
1100     // ...
1101     // ...
1102     // ...
1103     // ...
1104     // ...
1105     // ...
1106     // ...
1107     // ...
1108     // ...
1109     // ...
1110     // ...
1111     // ...
1112     // ...
1113     // ...
1114     // ...
1115     // ...
1116     // ...
1117     // ...
1118     // ...
1119     // ...
1120     // ...
1121     // ...
1122     // ...
1123     // ...
1124     // ...
1125     // ...
1126     // ...
1127     // ...
1128     // ...
1129     // ...
1130     // ...
1131     // ...
1132     // ...
1133     // ...
1134     // ...
1135     // ...
1136     // ...
1137     // ...

```

(b) usertests.c changes

Figure 3: User-space changes for collecting execution time, context switches, and process-state differences.

```

19 static void freeproc(struct proc *p);
20
21 extern char trampoline[]; // trampoline.S
22
23 // helps ensure that wakeups of wait()ing
24 // parents are not lost. helps obey the
25 // memory model when using p->parent.
26 // must be acquired before any p->lock.
27 struct spinlock wait_lock;

```

```

19 static void freeproc(struct proc *p);
20
21 extern char trampoline[]; // trampoline.S
22
23 extern uint64 dynamic_tick_rate;
24 uint64 total_context_switches = 0;
25 uint64 proc_running = 0;
26 uint64 proc_sleeping = 0;
27 uint64 proc_runnable = 0;
28 uint64 proc_unused = 0;
29 uint64 proc_used = 0;
30 uint64 proc_zombie = 0;
31
32 // helps ensure that wakeups of wait()ing
33 // parents are not lost. helps obey the
34 // memory model when using p->parent.
35 // must be acquired before any p->lock.
36 struct spinlock wait_lock;

```

(a) proc.c counter variables

```

541 // }
542 uint64 run_started_at = r_time();
543
544 // if(proc_sleeping > 5)
545 //   dynamic_tick_rate = 100000; // 0.1 second
546 // }
547 // else if (total_run_proc < 16)
548 //   dynamic_tick_rate = 2000; // 2 ms
549 // else
550 //   dynamic_tick_rate = 1000; // 1 ms
551
552 p->state = RUNNING;
553 c->proc = p;
554 switch(&c->context, &p->context);
555 total_context_switches++;
556 // uint64 run_time = r_time() - run_started_at;
557 // if(run_time > dynamic_tick_rate && proc_runnable_current > 1 && dynamic_tick_rate > 1000000) {
558 //   // uint64 next_tick_rate = dynamic_tick_rate / 2;
559 //   dynamic_tick_rate = next_tick_rate < 100000 ? 100000 : next_tick_rate;
560 // }
561
562 uint64 run_time = r_time() - run_started_at;
563 if (run_time > dynamic_tick_rate &&
564     proc_runnable_current > 10 &&
565     dynamic_tick_rate > 250000) {
566   uint64 next_tick_rate = (dynamic_tick_rate * 3) / 4;
567   dynamic_tick_rate = next_tick_rate < 250000 ? 250000 : next_tick_rate;
568 } //only decrease the tick rate if the runnables are too many. We want to be more resp
569 //but we don't want to decrease the tick rate if there are only a few runnable process
570 //below logic made to slowly increase the interval.
571 else if (proc_runnable_current <= 2) {
572   if (dynamic_tick_rate < 1000000 && proc_runnable_current == 2) {
573     uint64 next_tick_rate = (dynamic_tick_rate * 5) / 4;
574     dynamic_tick_rate = next_tick_rate > 1000000 ? 1000000 : next_tick_rate;
575   } else {
576     dynamic_tick_rate = 1500000;
577   }
578 }

```

(b) proc.c adaptive tick logic

Figure 4: Scheduler-side changes in proc.c.

```

9 struct spinlock tickslock;
10 uint ticks;
11
12
13

```

```

9 struct spinlock tickslock;
10 uint ticks;
11 uint64 dynamic_tick_rate = 1000000;
12 uint64 total_ticks = 0;
13

```

(a) trap.c variable initialization

```

163 void
164 clockintr()
165 {
166   if(cpuid() == 0){
167     acquire(&tickslock);
168     ticks++;
169     wakeup(&ticks);
170     release(&tickslock);
171   }
172
173   // ask for the next timer interrupt. this also clears
174   // the interrupt request. 1000000 is about a tenth
175   // of a second.
176   w_stimecmp(r_time() + 1000000);
177 }
178
179 // check if it's an external interrupt or software interrupt,

```

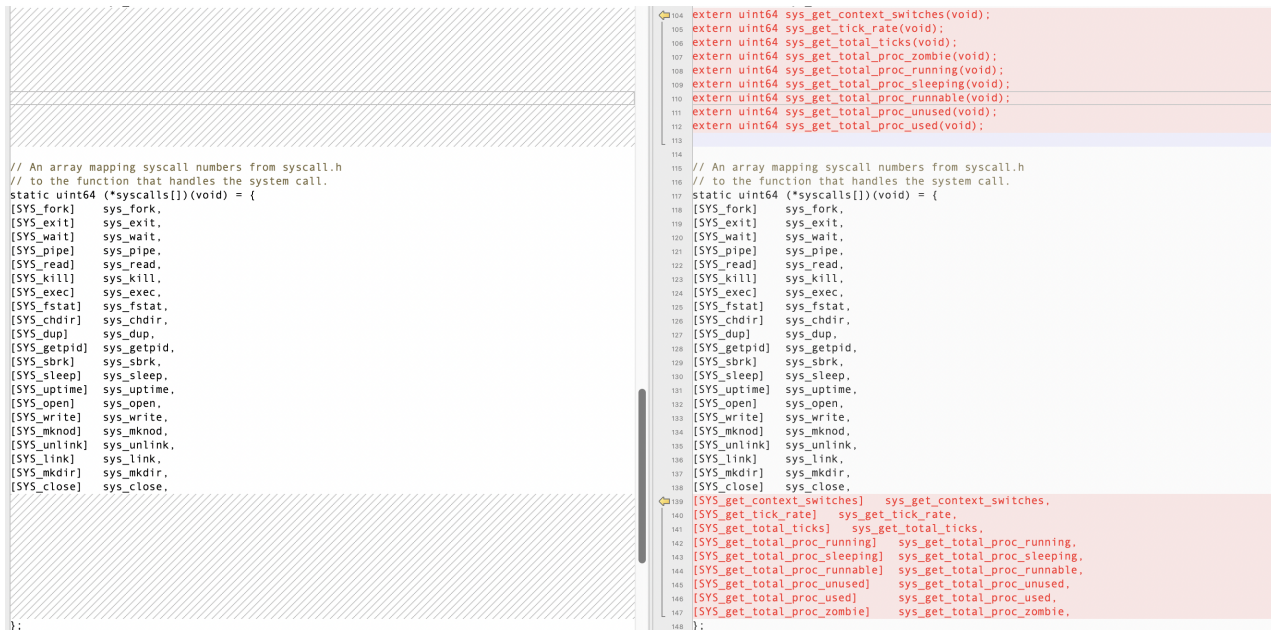
```

165 void
166 clockintr()
167 {
168   if(cpuid() == 0){
169     acquire(&tickslock);
170     ticks++;
171     total_ticks += dynamic_tick_rate;
172     wakeup(&ticks);
173     release(&tickslock);
174   }
175
176   // ask for the next timer interrupt. this also clears
177   // the interrupt request. 1000000 is about a tenth
178   // of a second.
179   //w_stimecmp(r_time() + 1000000);
180   w_stimecmp(r_time() + dynamic_tick_rate);
181 }
182
183 // check if it's an external interrupt or software interrupt,

```

(b) trap.c timer update path

Figure 5: Timer-interrupt changes for dynamic tick support.



(a) syscall.c entries



(b) syscall.h numbers

Figure 6: System-call wiring added to expose kernel metrics to user space (Part 1).

```

84 uint64
85 sys_uptime(void)
86 {
87     uint xticks;
88
89     acquire(&tickslock);
90     xticks = ticks;
91     release(&tickslock);
92     return xticks;
93 }

```

```

84 uint64
85 sys_uptime(void)
86 {
87     uint xticks;
88
89     acquire(&tickslock);
90     xticks = ticks;
91     release(&tickslock);
92     return xticks;
93 }
94
95 uint64
96 sys_get_context_switches(void)
97 {
98     extern uint64 total_context_switches;
99     return total_context_switches;
100 }
101
102 uint64
103 sys_get_tick_rate(void)
104 {
105     extern uint64 dynamic_tick_rate;
106     return dynamic_tick_rate;
107 }
108
109 uint64
110 sys_get_total_ticks(void)
111 {
112     extern uint64 total_ticks;
113     return total_ticks;
114 }
115
116 uint64
117 sys_get_total_proc_running(void)
118 {
119     extern uint64 proc_running;
120     return proc_running;
121 }
122
123 uint64
124 sys_get_total_proc_sleeping(void)
125 {
126     extern uint64 proc_sleeping;
127     return proc_sleeping;
128 }

```

(c) sysproc.c handlers

```

1 int sleep(int);
2 int uptime(void);

```

```

23 int sleep(int);
24 int uptime(void);
25 int get_context_switches(void);
26 int get_tick_rate(void);
27 int get_total_ticks(void);
28 int get_total_proc_running(void);
29 int get_total_proc_sleeping(void);
30 int get_total_proc_runnable(void);
31 int get_total_proc_unused(void);
32 int get_total_proc_used(void);
33 int get_total_proc_zombie(void);

```

```

34
35 // ulib.c
36 int stat(const char*, struct stat*);
37 char* strcpy(char*, const char*);
38 void *memmove(void*, const void*, int);
39 char* strchr(const char*, char c);

```

(d) user.h declarations

```

3 # Generate usys.S, the stubs for syscalls.
4
5 print "# generated by usys.pl - do not edit\n";
6
7 print "#include \"kernel/syscall.h\"\n";
8
9 sub entry {
10     my $name = shift;
11     print ".global $name\n";
12     print "$name:\n";
13     print "    li a7, SYS_$name\n";
14     print "    ecall\n";
15     print "    ret\n";
16 }
17
18 entry("fork");
19 entry("exit");
20 entry("wait");
21 entry("pipe");
22 entry("read");
23 entry("write");
24 entry("close");
25 entry("kill");
26 entry("exec");
27 entry("open");
28 entry("mkdir");
29 entry("unlink");
30 entry("fstat");
31 entry("link");
32 entry("mkdir");
33 entry("chdir");
34 entry("dup");
35 entry("getpid");
36 entry("sbrk");
37 entry("sleep");
38 entry("uptime");

```

```

2 # Generate usys.S, the stubs for syscalls.
3
4 print "# generated by usys.pl - do not edit\n";
5
6 print "#include \"kernel/syscall.h\"\n";
7
8 sub entry {
9     my $name = shift;
10    print ".global $name\n";
11    print "$name:\n";
12    print "    li a7, SYS_$name\n";
13    print "    ecall\n";
14    print "    ret\n";
15 }
16
17
18 entry("fork");
19 entry("exit");
20 entry("wait");
21 entry("pipe");
22 entry("read");
23 entry("write");
24 entry("close");
25 entry("kill");
26 entry("exec");
27 entry("open");
28 entry("mkdir");
29 entry("unlink");
30 entry("fstat");
31 entry("link");
32 entry("mkdir");
33 entry("chdir");
34 entry("dup");
35 entry("getpid");
36 entry("sbrk");
37 entry("sleep");
38 entry("uptime");
39 entry("get_context_switches");
40 entry("get_total_ticks");
41 entry("get_total_proc_running");
42 entry("get_total_proc_sleeping");
43 entry("get_total_proc_runnable");
44 entry("get_total_proc_unused");
45 entry("get_total_proc_used");
46 entry("get_total_proc_zombie");

```

(e) usys.pl stubs

Figure 7: System-call wiring added to expose kernel metrics to user space (Part 2).